Let’s apply this design principle and capture the dynamics with a state diagram, as shown in figure 3.5. This FSM is composed of

- *States*, including a starting state and one or more ending states, indicated by double circles by convention
- *Transitions* that take you from one state to another
- *Inputs* that bring about the transitions (T=0, T+10 days, T+11 days)
- Zero or more *outputs* during transitions. Registration (`Regs`), voting (`Vote`), and counting (`Done`), for example, happen in the states indicated.

Earlier, we defined four phases or states representing the ballot problem. These four phases are `Init`, `Regs`, `Vote`, and `Done`. The system begins its operation after initialization in the `Init` phase and then transitions into the `Regs` phase, where registration can take place. After ten days (in this case) for registration, the system moves to the `Vote` phase, in which voting goes on for a day, and then enters the (ballot) `Done` phase, at which point the winning proposal can be requested. Transitions in this case are temporal, or time-driven; you can see this in figure 3.5, with T=0, T+10, and T+11 limiting the duration of each phase. These dynamic rules for transitioning through the balloting process have to be captured when you code the smart contract to enable trust.